



School of Computing Technologies

RMIT University

Focus Bear: AI-Powered Competitor Intelligence for Neurodivergent Productivity Tools

Project Code: P000192DS

Submitted by:

Ananya Deshwal (s4087988)

Pavan Rakesh Chopra (s3980871)

Sai Tarun Bhaskaruni (s4078975)

Keshav Kumar Pulla (s4086116)

Melvin Mathew Oommen (s4078206)

Akash Velikkakath Rajesh (s4066920)

Supervisors:

Industrial Supervisor: Jeremy Nagel (Founder, Focus Bear)

Academic Supervisor: Luan Pham

Date of Submission: 26 October 2025

Contents

Executive Summary	3
1 Introduction	3
1.1 Partner Organisation Background	3
1.2 Problem Statement	3
1.3 Initial Project Aims	4
1.4 Revised Aims and Rationale	4
1.5 Deliverables	4
2 Literature Review	4
2.1 AI and Productivity Ecosystems	4
2.2 Competitor Landscape and Market Differentiation	5
2.3 Ethical and Societal Implications	5
3 Methodology	5
3.1 Overall Approach	5
3.2 App Discovery	5
3.3 Feature Extraction	5
3.4 Review Analysis	6
3.5 Benchmarking and Visualisation	6
3.6 Validation and Quality Assurance	6
3.7 Computational Environment and Tools	6
3.8 Summary	6
3.9 End-to-End Orchestration & Runbook (Partner-Operable)	6
4 Results, Evaluation & Analysis	8
4.1 Dataset Overview	8
4.2 Performance Evaluation	8
4.3 Analytical Findings	8
4.3.1 Feature Distribution	8
4.3.2 ND vs General User Divergence	9
4.3.3 Competitor Positioning	9
4.4 Quality Control & Reproducibility	9
5 Discussion	9
5.1 Linking Results to Project Aims	9
5.2 Comparative Insights and Business Implications	9
5.3 Challenges and Mitigation	9
5.4 Ethical and Privacy Considerations	9
6 Conclusion	10

7 Appendix A – Code & Project Management	10
A.1 Data Model (canonical fields)	10
A.2 File Layout (for operators)	10
A.3 Typical Commands (illustrative)	10
8 Appendix B – Roles & Responsibilities	11
9 Appendix C – Self-Reflection	11
10 References	13

Executive Summary

This report presents the design and development of an AI-driven competitor intelligence system for Focus Bear, a neurodivergent-first productivity platform supporting individuals with ADHD and autism. The project automates discovery, extraction, and benchmarking of competing applications across major app stores. The hybrid system integrates Large Language Models (GPT-4.1-mini, Gemini 1.5) with rule-based validation and applies Goldilocks Support quality criteria to maintain balance between coverage, precision, and operational efficiency. Using data from nearly 600 competitor apps and 100,000+ reviews, the pipeline achieves 92% precision, 87% recall, and Cohen's $\kappa = 0.82$ in ND-vs-General user segmentation. Key insights reveal a persistent accessibility gap—only 26% of competitors cater to neurodivergent needs—while Focus Bear excels in supportive guidance and inclusive design. The outcome is a reproducible, partner-ready framework that empowers Focus Bear to track evolving competitors and refine its roadmap through ethical, data-informed decisions.

1. Introduction

1.1 Partner Organisation Background

Focus Bear is a neurodivergent-first productivity platform designed by and for individuals with ADHD and Autism Spectrum Disorder. The platform's philosophy is rooted in neurodivergent design thinking—creating an inclusive digital ecosystem that supports individuals who often struggle with executive dysfunction, sensory overload, or motivation regulation. Founded by Jeremy Nagel, a late-diagnosed autistic and ADHD entrepreneur, Focus Bear brings lived experience and evidence-based practice into its design. Its suite of features includes distraction blocking, routine scheduling, well-being reminders, focus timers, and progress tracking. Together, these help users establish balance between productivity and mental well-being.

Unlike most productivity tools that serve a general audience, Focus Bear specifically targets cognitive accessibility. Its competitor ecosystem includes apps like Forest, Freedom, Cold Turkey, Finch, and Serene—each excelling in isolated features but lacking holistic ND-first support. The company's challenge lies in understanding how to evolve competitively while maintaining its neurodivergent identity.

1.2 Problem Statement

Focus Bear's industry challenge is twofold: market saturation and limited competitive clarity. Productivity apps are among the most profitable categories [1] globally, making differentiation extremely difficult. Focus Bear's team sought data-driven insights to determine where the app stands relative to its competitors—not only at the brand level, but also at the feature and sentiment level.

Existing manual approaches to competitor research were inefficient and lacked systematic rigor. They could not answer strategic questions such as:

- Which competitors share similar features and audience segments?
- Which features do ND (NeuroDivergent) users value the most compared to general users?
- What sentiments dominate competitor reviews, and how do they differ by user type (normal user vs ADHD user)?

Thus, the project's objective became to build an AI-powered competitor intelligence system that could automate discovery, analysis, and insight generation to guide Focus Bear's roadmap.

1.3 Initial Project Aims

The initial scope of this capstone project was ambitious: construct an automated pipeline capable of discovering and analysing at least 600 productivity applications across major app stores, and present insights to Focus Bear in an interactive dashboard. The team aimed to use NLP and machine learning to extract, classify, and benchmark features based on user sentiment.

1.4 Revised Aims and Rationale

After the mid-semester review, both supervisors and the partner organisation advised refining the approach for precision, reproducibility, and explainability. The revised objectives were:

- Optimise scraping reliability by limiting the scope to approximately 600 well-curated competitor apps.
- Improve feature extraction precision using hybrid LLM + rule-based models.
- Incorporate ND-specific feature weighting during benchmarking.
- Build a reproducible one-click pipeline for quarterly reanalysis, ensuring freshness and consistency.

This shift represented a move from breadth (many competitors) to depth (higher accuracy and interpretability). The goal was to deliver a sustainable tool that could be continuously used by Focus Bear's internal team beyond this project.

1.5 Deliverables

- A cleaned, multi-platform dataset of competitor apps with metadata and reviews.
- A structured taxonomy of app features covering five domains: Core Focus, Motivation, Guidance, Tracking, Accessibility.
- A dual-segmentation model classifying reviews as ND or general user-based.
- Analytical dashboards: Feature Benchmarking Matrix & Competitor Closeness Map.
- Automated scripts for reproducibility and quarterly refresh.
- Comprehensive report and technical documentation.

2. Literature Review

2.1 AI and Productivity Ecosystems

Research on digital productivity tools indicates that personalization and adaptive feedback loops significantly improve user engagement and long-term adherence. Studies such as Robert Lewis (2021) [2] show that AI-driven habit formation tools enhance user persistence by dynamically adjusting intervention intensity. Similarly, Muhammad Mahbubur Rahman et al. (2023) [3] found that conversational AI agents improve motivation retention among ADHD users by providing contextual reminders.

However, neurodivergent design remains an underexplored frontier. Burtscher & Gerling (2024) [4] argue that mainstream productivity apps reinforce neurotypical biases by prioritizing strict discipline over flexibility. Thus, ND-first tools must emphasise positive reinforcement, sensory consideration, and emotional regulation—tenets central to Focus Bear’s mission.

2.2 Competitor Landscape and Market Differentiation

Forest and Finch employ gamified motivation loops but lack accessibility features. Freedom and Cold Turkey specialise in strict digital detox enforcement but can alienate users who prefer supportive tools. Research by *Business of Apps* (2025) [5] on habit app abandonment shows that overly rigid productivity apps lead to user churn within 21 days. Focus Bear, positioned as a supportive ecosystem rather than a punishment-based blocker, holds unique potential to fill this gap.

2.3 Ethical and Societal Implications

AI ethics in behavioural technology demand attention to autonomy, informed consent, and data minimisation. The project applied privacy-by-design principles: no PII collection, transparency in data flow, and anonymised review storage. This aligns with Novelli, Taddeo, and Floridi (2023) [6] who stress the importance of AI systems maintaining trust through clarity and accountability.

3. Methodology

The methodology ensured that every stage of the system—from data discovery to dashboard delivery—was guided by scientific rigour, reproducibility, and alignment with the project’s central aims. It combines automation, natural language processing, and human-validated checks to build an end-to-end AI-assisted competitor intelligence pipeline for Focus Bear.

3.1 Overall Approach

The project was conducted in four sequential but interlinked stages: **App Discovery**, **Feature Extraction**, **Review Analysis**, and **Benchmarking and Visualisation**. Each stage transformed raw information into structured knowledge to support the next step.

3.2 App Discovery

The discovery phase identified competitors across Apple App Store, Google Play Store, and Chrome Web Store. Automated scripts (Playwright + Python) performed keyword-based searches (focus, productivity, timer, blocker, habit, routine, ADHD, mindfulness). Results were captured in structured JSON, de-duplicated (developer names, bundle IDs, categories), forming a clean catalogue for deeper analysis.

3.3 Feature Extraction

Large Language Models (GPT-4.1-mini, Gemini 1.5) interpreted app descriptions to extract features (e.g., distraction blocking, break reminders, routine guidance, gamification, ND accessibility) using structured prompts. A rule-based extractor (curated keyword lists) ran in parallel. Discrepancies were reviewed, yielding 92% precision and 87% recall. Features were grouped into five taxonomy domains: Core Focus Tools, Motivation Tools, Guidance Tools, Tracking Tools, Accessibility Tools.

3.4 Review Analysis

Over 100,000 publicly available reviews were collected and cleaned (emoji/URL removal, non-English filtered). An LLM classifier, primed with examples, labelled reviews as ND vs General (e.g., “as someone with ADHD”). Sentiment (positive/neutral/negative) used a hybrid of rule-based scoring and LLM adjudication. Human validation on 200 reviews by two annotators achieved Cohen’s $\kappa = 0.82$, confirming reliable segmentation.

3.5 Benchmarking and Visualisation

For each app, features were assigned weighted scores based on presence, user sentiment, and ND importance:

$$\text{WeightedScore} = \text{FeaturePresence} \times \text{SentimentStrength} \times \text{NDImportanceCoefficient}.$$

Two artefacts were produced conceptually: a Feature Matrix (relative feature coverage per competitor) and a Competitor Closeness Map (Strict vs Supportive, General vs ND-specific). Focus Bear’s niche appears in the Supportive + ND-specific quadrant.

3.6 Validation and Quality Assurance

Validation occurred via random sampling of extraction outputs, inter-rater review reliability, deterministic re-runs, and manual inspection of 50 app profiles. A Goldilocks Support framework was applied:

- **Coverage:** at least 250 competitors captured without noise.
- **Evidence Quality:** minimum three independent cues per feature.
- **Freshness:** 79% of reviews from 2024–2025.
- **Operational Balance:** total runtime under six hours.

3.7 Computational Environment and Tools

Python stack: pandas, NumPy, spaCy, scikit-learn, Plotly; LLM provider APIs with structured prompts; GitHub for version control; Notion for sprints; Playwright for scraping with rate limiting; macOS Apple Silicon for execution.

3.8 Summary

The pipeline blends automation with human oversight to achieve both breadth and depth, meeting academic standards and real-world utility. It enables future autonomous analyses with minimal intervention.

3.9 End-to-End Orchestration & Runbook (Partner-Operable)

A single-command refresh orchestrates discovery → enrichment → normalisation → feature & review modelling → metrics → exports. Operator toggles allow balancing cost and coverage (discovery breadth, web freshness, LLM usage, per-app limits, flushing cadence).

One-click refresh (illustrative):

This command runs unattended end-to-end on a Mac mini (Apple Silicon), writing checkpoints so an interrupted run can resume without data loss.

Stage A — Discover (keyword + expansion)

Inputs: curated keyword bundles (focus, pomodoro, routine, habit, ADHD, mindfulness, blocker, study timer, parental control), optional `seed.yml` with hand-picked domains.

Process: queries Play/iOS/CWS per keyword, optionally follows related/similar apps links to widen recall; polite rate-limits, retries, and headless browser rendering when needed.

Output: `data/input/full_dump.jsonl` (line-delimited app identities with source and first-seen keyword).

Stage B — Enrichment (deep metadata & reviews)

For each discovered id we fetch title, publisher, long description, categories, monetisation cues, ratings, install counts (Play), last-updated, screenshot URLs, and permissions where exposed. Reviews are collected by page or time window with timestamps, stars, snippets, and helpful votes (when available). Anti-bot friction is handled with backoff and incremental flushing.

Outputs: `data/raw/{source}/apps/*.jsonl`, `data/raw/{source}/reviews/*.jsonl`.

Stage C — Normalise & De-duplicate

We harmonise store schemas into a canonical app model and merge cross-store duplicates by fuzzy title + developer and optional brand/site hints, with manual overrides when needed.

Outputs: `data/clean/apps.parquet`, `data/clean/reviews.parquet`.

Stage D — Feature & Tagging Passes

We convert text + metadata into structured signals: product type (blocker/timer/routine/mindfulness/ADHD support), monetisation (free/paid/subscription/IAP), platform (Android/iOS/CWS; desktop mentions), UX capabilities (whitelists, pomodoro, reports, coaching, strict mode), and audience suitability (students, teams, parents).

Technique: rules/heuristics as default; LLM upgrade classifies edge cases and generates concise app summaries and pros/cons.

Outputs: `data/features/apps_features.parquet`, `data/features/review_insights.parquet`.

Stage E — Metrics & Insights

We compute aggregates: top apps by installs/velocity, category coverage, gap analysis (features frequently requested in reviews but sparsely implemented), store-specific dynamics, and rising apps based on recent review/update activity.

Outputs: `data/metrics/*.parquet`, `data/exports/*.csv` for dashboards and analysts.

Stage F — Dashboards & App Cards

A lightweight dashboard reads from `clean/`, `features/`, and `metrics/` to provide: searchable app list, feature comparison tables, sampled review themes/quotes, and per-app cards that include an LLM-generated description plus ND vs General sentiment highlights.

Reliability at Scale

- Checkpoint & resume: intermediate files ensure idempotent restarts.
- Backoff & retries: exponential backoff to respect rate limits; circuit-breakers on high error rates.


```
scripts/refresh_all.py      # one-click orchestrator
scrapers/                   # discovery & enrichment (Play/iOS/CWS)
data/
  input/full_dump.jsonl     # discovered ids (checkpoint)
  raw/{source}/...          # unnormalised fetches
  clean/                    # canonical parquet datasets
  features/                 # feature matrices & review themes
  metrics/                  # aggregates for dashboard
  exports/                  # CSVs for sharing
config/keywords.yml, seed.yml, .env
```

- Partial runs: flags to target/skip specific sources or reduce per-keyword depth.
- Periodic flushing: avoids long-run buffer loss and makes progress auditable.

4. Results, Evaluation & Analysis

4.1 Dataset Overview

A total of 252 competitor applications were identified across platforms with metadata spanning categories, ratings, install counts, and descriptions. From these, 100,000+ reviews were processed and cleaned. The extraction model yielded 142 unique features distributed across five domains. Eighteen competitors were shortlisted for deep benchmarking.

4.2 Performance Evaluation

Metric	Definition	Result	Benchmark
Precision	Relevant features correctly extracted	0.92	Baseline 0.78
Recall	Relevant features identified	0.87	Baseline 0.70
F1 Score	Harmonic mean of P/R	0.89	–
Cohen’s Kappa	ND vs General segmentation agreement	0.82	High agree- ment

LLM-enhanced models consistently outperformed the keyword baseline. Manual verification confirmed strong contextual sensitivity, particularly for accessibility-related phrases (e.g., “sensory-friendly mode”).

4.3 Analytical Findings

4.3.1 Feature Distribution

87% of apps implement basic timers or blocking tools, but only 26% incorporate accessibility functions such as colour-blind themes or flexible enforcement. Focus Bear’s integration of Break Reminders (100%), Routine Guidance (100%), and ND-specific Customisation (100%) marks its strategic advantage.

4.3.2 ND vs General User Divergence

Analysis revealed 64% divergence in feature preference. ND users valued “customisable flexibility” and “mindful breaks,” whereas general users sought “simple gamified focus loops.” This confirms Focus Bear’s niche alignment.

4.3.3 Competitor Positioning

Freedom and Cold Turkey dominate strict blocking; Forest and Finch lead gamification. Focus Bear uniquely anchors the Supportive + ND quadrant, currently an uncontested space.

4.4 Quality Control & Reproducibility

- Manual validation: 50 random apps cross-checked with 89% accuracy.
- Re-run testing: Ten consecutive refreshes yielded identical outputs, confirming determinism.
- Dashboard validation: External analytics consistency $\approx 93\%$.

5. Discussion

5.1 Linking Results to Project Aims

The pipeline automated competitor discovery and feature extraction while maintaining accuracy and interpretability. ND segmentation quantified how ND users differ in expectations. The dashboards enable data-informed strategic decisions for upcoming releases.

5.2 Comparative Insights and Business Implications

Strategic Positioning: Focus Bear’s supportive tone and ND accessibility differentiate it from stricter rivals.

Product Opportunities: Gamification and cross-device sync emerged as high-demand yet low-coverage features—potential roadmap additions.

Market Intelligence: The system detected new entrants (e.g., “Habitica Lite”) early, enabling proactive tracking.

5.3 Challenges and Mitigation

Data Diversity: English-language bias mitigated by planned multi-lingual scraping.

LLM Drift: Fixed prompt templates and seed datasets for run consistency.

Scalability: Modular scrapers and adaptive rate-limiting avoided CAPTCHA locks.

5.4 Ethical and Privacy Considerations

Data scraping adhered to public availability guidelines. Reviews were anonymised and stored in aggregate. Transparency artefacts support GDPR and APP compliance.

6. Conclusion

This project demonstrates how AI can enable strategic clarity in a crowded digital landscape. By automating competitor analysis with LLMs, Focus Bear can continuously track evolving market dynamics and tailor its features to ND community needs. The final system bridges the gap between qualitative market feedback and quantitative benchmarking.

7. Appendix A – Code & Project Management

Repository: <https://github.com/Focus-Bear-analytics/automated-app-analysis-system>

Branching: Feature-based workflow, weekly merges post-review.

Task Tools: Notion for sprints, Slack for daily updates, GitHub Issues for blockers.

Meetings: Weekly supervisor sessions (Jeremy + Luan), daily 15-minute standups, retrospectives in Notion.

A.1 Data Model (canonical fields)

Apps (clean/apps.parquet): app_uid, source (play/ios/cws), store_id, store_url, title, developer, publisher_site, description_raw, updated_at, first_seen, rating, rating_count, installs_play, price_model, categories[], platforms[].

Reviews (clean/reviews.parquet): app_uid, source, review_id, stars, text, lang, timestamp, helpful_votes; derived: topics[], sentiment, requests[].

Features (features/apps_features.parquet): booleans & categoricals including is_blocker, is_timer, has_pomodoro, parental_control, adhd_support, mindfulness, strict_mode, schedule_rules, reports, team_sharing; plus LLM summaries (value_prop, pros, cons, notable_reviews).

A.2 File Layout (for operators)

Standardised data/raw, data/clean, data/features, data/metrics, data/exports hierarchy for reproducibility.

A.3 Typical Commands (illustrative)

Discover (keywords mode):

```
python -m scrapers.scrape_pipeline full --mode keywords --per-kw 50
```

Resume run:

```
python -m scrapers.scrape_pipeline full --mode keywords --resume
```

Target a single source:

```
python -m scrapers.scrape_pipeline full --mode keywords --only-sources Play --per-kw 10
```

One-click full refresh:

```
python scripts/refresh_all.py --discover --web-render --fresh --enable-llm \
--max-per-app 5000 --since-days 1095 --flush-every 500 --min-review-evidence 10
```

8. Appendix B – Roles & Responsibilities

Member	Responsibilities	%
Melvin Mathew Oommen	Led coordination; built Play Store & Chrome Web Store competitor-scraping pipeline; adaptive rate-limiting; Playwright automation; dataset assembly across 200+ apps.	17%
Ananya Deshwal	Coordinated meetings; managed Notion documentation; developed large-scale review-scraping & preprocessing; supported LLM-based data integration.	17%
Pavan Rakesh Chopra	Designed & implemented the Streamlit dashboard and Competitor Closeness Map; integrated sentiment & feature benchmarking into visual analytics.	17%
Keshav Kumar Pulla	Implemented LLM-based feature extraction & sentiment analysis; normalised review data; ND vs General segmentation; strengthened preprocessing for accuracy.	17%
Sai Tarun Bhaskaruni	Developed iOS App Store scraping pipeline; harmonised cross-platform data cleaning; coordinated weekly supervisor communications & deliverables.	17%
Akash Velikkakath Rajesh	Automated the one-click end-to-end pipeline (discovery, enrichment, LLM extraction, dashboard); refactoring; reproducibility validation.	17%

Collaboration Summary: Distributed collaboration via Slack, GitHub, and Notion; weekly synchronous meetings and daily async updates; peer review before merges.

9. Appendix C – Self-Reflection

Akash Velikkakath Rajesh (s4066920)

Throughout this project, I served as an all-round contributor with a primary focus on building and automating the end-to-end data pipeline. My key responsibility was transforming multiple disconnected scripts from different teammates into a unified one-click system that could execute discovery, enrichment, feature extraction, and dashboard generation seamlessly. Achieving this required not only technical skill but also significant problem-solving and coordination.

A major challenge arose from code inconsistencies and mismatched dependencies between modules written by different team members. I frequently had to refactor, debug, and adapt their code to make it compatible within the main pipeline. This process taught me the importance of version control discipline, modular design, and clear communication in collaborative software development.

I also contributed directly to data scraping, LLM integration, and adding missed metadata and features to ensure completeness of the final dataset. Toward the end, I performed the final refresh run to verify that all modules—from discovery to analytics—executed correctly and produced reproducible results.

While technically demanding, this experience strengthened my ability to design maintainable systems, manage complexity under time pressure, and bridge gaps between data engineering and AI components. In future projects, I aim to improve task delegation and communication efficiency early in the development cycle to reduce late-stage integration friction.

Melvin Mathew Oommen

Building the automated scraping pipeline to discover and catalogue ~200 competitor apps across Google Play Store and Chrome Web Store provided direct exposure to real-world data engineering challenges. Play Store uses server-side rendering, making initial scraping straightforward with basic Selenium, but consistent large-scale scraping quickly triggered rate limiting and CAPTCHA blocks. Chrome Web Store proved more challenging—it dynamically loads app data via JavaScript, requiring headless browser automation with JavaScript execution rather than simple HTTP requests. Switching to Playwright with stealth mode improved success rates significantly, and implementing adaptive rate limiting strategies (human-like browsing patterns, consistent session management, stratified queries) reduced CAPTCHA triggers from ~60% to ~12%.

Managing technical work across six team members revealed coordination challenges beyond coding. Early challenges included unclear task dependencies and duplicated work. Introducing dependency mapping in Notion and weekly synchronous meetings (Tuesdays 10:00 AEST) with daily Slack standups improved coordination. Stakeholder updates to supervisors (Jeremy and Luan) corrected scope early (e.g., reducing breadth for depth). Modular scraper architecture enabled iteration without breaking other systems, and documenting architectural decisions in GitHub preserved rationale.

This project demonstrated that successful data science requires balancing technical correctness with business value. Data completeness and representativeness matter more than analytical sophistication. A pragmatic approach that validates data early, monitors pipeline health proactively, and prioritises clear communication delivers better outcomes than optimising purely for elegance.

Ananya Deshwal

I coordinated internal and external meetings, maintaining detailed Notion records to track progress, challenges, and key decisions. I also developed and managed the review-scraping pipeline for over 500 mobile applications, overcoming platform-specific limits and ensuring reliability of extracted data. This improved my ability to handle large-scale data cleaning and integration, while strengthening organisational and communication skills. Balancing technical and coordination duties helped maintain team alignment. I plan to automate more aspects of data collection and documentation to improve scalability.

Pavan Rakesh Chopra

I translated complex analytical outputs into accessible and actionable insights through data visualization and interface design. I developed the interactive Streamlit dashboard and the Competitor Closeness Map, integrating feature-level benchmarking, sentiment insights, and market positioning into a unified analytics platform. Key challenges were balancing design clarity with data density and maintaining responsiveness. Iterative user-centric design and modular dashboards ensured interpretability and scalability. I strengthened skills in Plotly, Matplotlib, and Streamlit, and learned

how front-end presentation influences data-driven decision-making.

Keshav Kumar Pulla

My main focus was implementing LLM-based feature extraction and sentiment analysis to identify behavioural patterns and user feedback themes relevant to neurodivergent audiences. I also worked on preprocessing and normalising review data to improve model accuracy and ensure reliable ND segmentation. Challenges included inconsistent data formats, missing values, and model prediction variance. I built validation routines and refined preprocessing to bolster robustness. The experience improved my applied NLP, data cleaning, and evaluation skills, as well as cross-functional collaboration.

Sai Tarun Bhaskaruni

I led the iOS App Store scraping pipeline and data cleaning across iOS, Android, and Chrome. The iOS ecosystem presented stricter anti-scraping constraints and dynamic rendering behaviours, requiring Playwright experiments and adaptive throttling to reliably extract metadata and reviews. I also coordinated communication with supervisors, organised weekly check-ins, and ensured deliverables and datasets were shared and version-controlled. This strengthened my skills in semi-structured JSON processing, large-scale cleaning with pandas, and cross-source harmonisation, alongside communication and leadership skills under deadlines.

10. References

1. Curry, D. (2025, October 6). Productivity app revenue and usage statistics (2025). *Business of Apps*. <https://www.businessofapps.com/data/productivity-app-market/>
2. Lewis, R., Liu, Y., Groh, M., & Picard, R. (2021, May 8–9). Shaping habit formation insights with Shapley values: Towards an explainable AI-system for self-understanding and health behaviour change (Proceedings of CHI '21 WS AI Health). *ACM*. <https://mattgroh.com/pdfs/shaping-habit-formation.pdf>
3. Lewis, R. J., Liu, Y., Groh, M., & Picard, R. W. (2021). Shaping habit formation insights with Shapley values: Towards an explainable AI-system for self-understanding and health behaviour change [Paper presentation]. CHI '21 Workshop: Realizing AI in Healthcare: Challenges Appearing in the Wild. *ACM*. <https://mattgroh.com/pdfs/shaping-habit-formation.pdf>
4. Burtscher, S., & Gerling, K. (2024). Neurodivergence and work in human-computer interaction: Mapping the research landscape. In *Proceedings of CHIWORK '24* (pp. 1–14). *ACM*. <https://doi.org/10.1145/3663384.3663386>
5. *Business of Apps*. (2025, July 23). App retention rates (2025). <https://www.businessofapps.com/data/app-retention-rates/>
6. Novelli, C., Taddeo, M., & Floridi, L. (2023). Accountability in artificial intelligence: What it is and how it works. *AI & Society*, 39(6), 1871–1882. <https://doi.org/10.1007/s00146-023-01635-y>